NLP Group Assignment 1 — Final Report

Ashutosh Kumar Jha - 12340390
Sidhesh Kumar Patra - 12342060
Ajay Chikate - 12340580
Aman Kumar - 12340190
Kishor Koppikar - 12341210

10-09-2026

Scope

This assignment consists of **four integrated Natural Language Processing (NLP) systems** spanning multiple fundamental NLP tasks, including:

- Word Segmentation
- Part-of-Speech (POS) Tagging
- Dependency Parsing
- Spelling Correction
- A Live Constituency-Aware Editor

The systems are designed to demonstrate practical implementations of different NLP techniques and their integration into interactive and functional applications.

Contents

1 Executive Summary

This report consolidates four related NLP subsystems developed as a single group assignment. Question 1 builds a **word segmenter and POS tagger** for unspaced English and Spanish text; Question 2 builds a **greedy arc-standard dependency parser**; Question 3 builds an **efficient spelling corrector** for non-word and real-word errors; and Question 4 **integrates all three** (segmentation, spelling correction, and a new PCFG constituency parser) into a live, background-running text editor. Across every task, the proposed statistically-grounded models substantially outperform simple deterministic or frequency-only baselines, and the integration work in Question 4 demonstrates that errors cascade strongly from segmentation into downstream stages — motivating the emphasis on high-fidelity segmentation across the whole assignment.

Question	System	Headline Result
Q1	Trigram LM + Viterbi segmentation; Trigram HMM POS tagging	English 96.45% / Spanish 90.40% segmentation Word F1; end-to-end pipeline accuracy 94.29% (EN) / 86.75% (ES)
Q2	Arc-standard dependency parser, logistic-regression oracle-trained	Dev-set LAS 56.84%, UAS 66.75%, ∼3,400 tok/s
Q3	SymSpell-style spelling corrector with bigram real-word disambiguation	81.03% non-word accuracy, 68.87% real-word accuracy, Method B ∼10.1x faster than brute force
Q4	Live editor integrating Q1 + Q3 + new PCFG parser	0.343 ms/word full pipeline latency; 86.4% agreement between real-time and end-of-passage grammar verdicts

Table 1: Executive summary of headline results across all four questions.

2 Question 1 — Word Segmentation & POS Tagging

2.1 Overview

This system performs end-to-end sentence segmentation and Part-of-Speech tagging on continuous, unspaced text in **English** (NLTK Brown Corpus) and **Spanish** (Universal Dependencies `UD_Spanish-GSD`). It contrasts statistically-grounded models against simple deterministic baselines and separately reports a morphology-aware tagging extension.

Datasets:

- English: Brown Corpus, 80/20 split — 35,176 training sentences, 8,795 test sentences.
- Spanish: `UD_Spanish-GSD` — 14,181 training sentences, 427 test sentences.

2.2 Architecture

1. **Word Segmentation Engine** — a trigram language model with linear interpolation smoothing ($\lambda_3 = 0.70$, $\lambda_2 = 0.20$, $\lambda_1 = 0.09$, $\lambda_0 = 0.01$), backed by a character-level subword LM for OOV tokens, decoded with Viterbi dynamic programming plus beam search.
2. **POS Tagging Engine** — a second-order trigram Hidden Markov Model, $P(t_i \mid t_{i-2}, t_{i-1})$, with linearly interpolated transition smoothing and Lidstone/suffix-smoothed emission probabilities.
3. **Morphology-Aware Extension** — fine-grained tags (e.g. `NOUN-Fem-Sing`, `ADJ-Masc-Plur`, `VERB-Sing3`) that encode grammatical agreement between adjacent tokens.
4. **Baselines** — a greedy longest-match dictionary segmenter, and a Most-Frequent-Tag (MFT) POS baseline.

Repository layout: `data_loader.py` (Brown/UD-GSD parsing), `segmentation.py` (trigram LM + Viterbi DP + baseline), `pos_tagger.py` (trigram HMM + morphology tags + MFT baseline), `pipeline.py` (joint inference), `evaluate.py` (metrics + error breakdown), `main.py` (CLI).

2.3 Results

Word segmentation (400 holdout sentences):

Language	Model	Precision	Recall	Word F1	Sentence Exact Match	Inference Time
English	**Trigram LM + DP**	**96.16%**	**96.73%**	**96.45%**	**69.25%**	12.53s
English	Greedy Longest-Match	63.24%	75.54%	68.85%	16.00%	0.02s
Spanish	**Trigram LM + DP**	**89.96%**	**90.86%**	**90.40%**	**30.00%**	19.44s
Spanish	Greedy Longest-Match	45.43%	58.83%	51.27%	4.75%	0.03s

Table 2: Word segmentation results on 400 holdout sentences.

POS tagging accuracy (gold-segmented tokens):

End-to-end pipeline and error-source breakdown:

Top confusions:

- **English:** VERB→NOUN (18.40% of confusions, zero-derivation homographs like *run*, *plan*); NOUN→VERB (10.38%); NOUN→ADJ (8.49%, attributive nouns); DET↔ADV and PRT↔ADP pairs (∼6% each, dual-function words / phrasal-verb particles).
- **Spanish:** PROPN→NOUN (17.10%, driven by loss of capitalization in unspaced text); NOUN→PROPN (7.26%); ADJ↔NOUN pairs (∼6–7%, postpositive/nominalized adjectives);

Language	Tagset / Model	Accuracy	Correct / Total	Tagset Size
English	Standard: Trigram HMM	**97.20%**	7,348 / 7,560	11
English	Standard: MFT baseline	94.48%	7,143 / 7,560	11
English	Morph-Aware: Trigram HMM	**97.02%**	7,335 / 7,560	24
English	Morph-Aware: MFT baseline	92.33%	6,980 / 7,560	24
Spanish	Standard: Trigram HMM	**93.94%**	9,605 / 10,225	16
Spanish	Standard: MFT baseline	89.09%	9,109 / 10,225	16
Spanish	Morph-Aware: Trigram HMM	**91.76%**	9,382 / 10,225	125
Spanish	Morph-Aware: MFT baseline	85.70%	8,763 / 10,225	125

Table 3: POS tagging accuracy on gold-segmented tokens.

Language	Pipeline	Accuracy	Total Errors	Segmentation-Induced	Genuine Tagging
English	Proposed (Standard)	**94.29%**	477	292 (61.2%)	185 (38.8%)
English	Baseline (Standard)	71.81%	3,601	3,319 (92.2%)	282 (7.8%)
English	Proposed (Morph-Aware)	94.27%	478	292 (61.1%)	186 (38.9%)
English	Baseline (Morph-Aware)	70.32%	3,714	3,319 (89.4%)	395 (10.6%)
Spanish	Proposed (Standard)	**86.75%**	1,457	1,037 (71.2%)	420 (28.8%)
Spanish	Baseline (Standard)	54.67%	7,651	7,226 (94.4%)	425 (5.6%)
Spanish	Proposed (Morph-Aware)	84.80%	1,656	1,037 (62.6%)	619 (37.4%)
Spanish	Baseline (Morph-Aware)	53.16%	7,805	7,226 (92.6%)	579 (7.4%)

Table 4: End-to-end pipeline accuracy and error-source breakdown.

PRON→DET (6.13%, *la/los/las* homophony); CCONJ→SCONJ (4.68%, polysemous *que/como*); VERB→AUX (4.52%, *ser/estar/haber* copula overlap).

2.4 Discussion

Where English and Spanish diverge. Spanish trails English on both segmentation (90.40% vs. 96.45% F1) and gold-token POS accuracy (93.94% vs. 97.20%). Two compounding factors explain this: (1) Spanish's richer inflectional morphology produces a higher type-to-token ratio (40,702 unique types over 14k training sentences vs. 34,729 types over 35k English sentences), increasing OOV rates; and (2) Spanish sentences average 25.5 tokens vs. 18.9 for English, so a per-token error rate compounds over more decisions before a sentence counts as an exact match (roughly $0.96^{19} \approx 0.46$ for English vs. $0.90^{25} \approx 0.07$ for Spanish). Spanish's largest POS error source — PROPN→NOUN — is a direct consequence of losing capitalization once text is unspaced and lowercased, forcing the tagger to rely purely on lexical and transition statistics.

Does morphology-aware tagging help or add noise? It is a net-negative on aggregate accuracy: English drops from 97.20% to 97.02% (−0.18%) and Spanish drops from 93.94% to 91.76% (−2.18%). The theoretical benefit — directly modeling agreement constraints such as $P(\text{ADJ-Fem-Sing} \mid \text{DET-Fem-Sing}, \text{NOUN-Fem-Sing}) \gg P(\text{ADJ-Masc-Plur} \mid \ldots)$ — is real, and the model does correctly enforce local concord (e.g. *una gran fiesta esperada* → consistent feminine-singular tags throughout). But expanding the Spanish tagset from 16 to 125 tags inflates the second-order transition space from $16^3 = 4{,}096$ to $125^3 \approx 1.95$M parameters, and even with interpolation smoothing, rare morphological combinations suffer from data sparsity that outweighs the agreement benefit at the aggregate level.

Segmentation as the dominant error source. Aligning predicted character spans against gold spans shows that 61.2% (English) and 71.2% (Spanish) of proposed-pipeline errors are segmentation-induced rather than genuine tagging mistakes; for the baseline this rises to 92–94%. A single boundary error cascades into the downstream tagger, so segmentation quality is the single largest lever on end-to-end accuracy — the proposed DP segmenter cuts segmentation-induced errors by over 85% relative to the greedy baseline.

Model vs. baseline gap. The proposed models beat baselines on every axis: segmentation F1 improves by +27.6 points (English) and +39.1 points (Spanish); pipeline accuracy improves by +22.5 points (English, 7.5x error reduction) and +32.1 points (Spanish, 5.2x error reduction). Qualitatively, the greedy baseline is vulnerable to "trapping" — on `thequickbrownfoxjumpsoverthelazydog` it locally matches `overt` (adjective), consuming the `t` of `the` and fragmenting the remainder into single-letter non-words, whereas the DP segmenter finds the globally coherent split. A parallel failure occurs in Spanish, where the baseline matches the verb *casar* inside `lacasarojaesgrande` instead of the noun *casa*.

2.5 Sample Outputs

- `thequickbrownfoxjumpsoverthelazydog` → `the/DET quick/ADJ brown/NOUN fox/NOUN jumps/VERB over/ADP the/DET lazy/ADJ dog/NOUN` (proposed); baseline mis-splits into `overt/ADJ hel/NOUN a/DET z/NOUN y/NOUN dog/NOUN`.
- `lacasarojaesgrande` → `la/DET-Fem-Sing casa/NOUN-Fem-Sing roja/ADJ-Fem-Sing es/AUX-Sing-P3 grande/ADJ-Sing`, correctly carrying feminine-singular concord across the whole noun phrase; baseline mis-splits *casar* (verb) + *oja*.
- `mispadrespuedenviajar` → `mis/DET-Plur padres/NOUN-Masc-Plur pueden/AUX-Plur-P3 viajar/VERB-I`
  correctly identifying plural agreement and the auxiliary + infinitive construction.

2.6 Conclusion

Dynamic programming over a smoothed trigram LM eliminates the greedy segmenter's local-minima traps, lifting Word F1 from 68.85% → 96.45% (English) and 51.27% → 90.40% (Spanish). Morphology-aware tagging demonstrably captures grammatical agreement but is outweighed by data sparsity at current corpus sizes, and the error-source breakdown confirms segmentation quality — not tagging quality — is the primary bottleneck in joint unspaced-text pipelines.

3 Question 2 — Transition-Based Dependency Parser

3.1 Overview

A greedy, arc-standard transition-based dependency parser trained on Universal Dependencies English-EWT, implemented as small single-purpose modules:

File	Responsibility
`conllu_io.py`	Parses `.conllu` files into `Sentence` objects (forms, UPOS, gold heads/labels)
`transition_system.py`	`Configuration` class + SHIFT / LEFT-ARC / RIGHT-ARC transitions
`oracle.py`	Static oracle: replays the gold tree through arc-standard to emit training instances
`features.py`	Extracts the 4 required POS-tag features from a configuration
`train.py`	Builds the training set and fits a scikit-learn classifier
`parser.py`	Greedy parsing loop driven by the trained classifier
`evaluate.py`	Computes UAS/LAS on the dev set
`main.py`	Runs the whole pipeline end to end

Table 5: Module responsibilities for the dependency parser.

3.2 Design Choices

Transition system. The stack starts as `[ROOT]` (id 0); the buffer holds all sentence tokens in order. LEFT-ARC pops `stack[-2]` (making `stack[-1]` its head) and is disallowed when `stack[-2]` is ROOT, since the root can never be a dependent. RIGHT-ARC pops `stack[-1]` (making `stack[-2]` its head).

Oracle. A classic static oracle (Nivre, 2004): LEFT-ARC fires if the gold head of `stack[-2]` is `stack[-1]`; RIGHT-ARC fires if the gold head of `stack[-1]` is `stack[-2]` *and* `stack[-1]` has already collected all its own gold children (to avoid orphaning them); otherwise SHIFT. Because arc-standard can only reproduce **projective** trees, sentences where the oracle gets stuck are skipped when building the training set — this affects 287 of 12,544 English-EWT training sentences (∼2.3%), consistent with the treebank's known non-projectivity rate.

Multiword tokens / empty nodes. CoNLL-U ranges (e.g. `8-9` for contractions) and empty nodes (e.g. `8.1` for elided material) are skipped during reading, keeping only tokens with plain integer IDs that receive a syntactic head.

Classifier. Each oracle step emits a `(transition, label)` pair, collapsed into one multi-class label (e.g. `LEFT-ARC:det`, `RIGHT-ARC:nsubj`, `SHIFT`) predicted jointly by a single scikit-learn `LogisticRegression` (`lbfgs`) over one-hot `DictVectorizer` features. With only 4 categorical POS-tag features (∼73 one-hot dimensions) and 82 output classes, training completes in under 2 minutes on the full training set.

Parsing loop / legality masking. At each step, the classifier's `predict_proba` ranking is walked until a *legal* transition is found (LEFT-ARC/RIGHT-ARC require ≥ 2 stack items; LEFT-ARC additionally requires `stack[-2] != ROOT`), guaranteeing the parser always terminates in a well-formed tree even when the raw classifier prefers an illegal move.

3.3 Data

- Training: `en_ewt-ud-train.conllu` — 12,544 sentences, 12,257 used after removing non-projective ones → **392,242** training instances.
- Evaluation: `en_ewt-ud-dev.conllu` — 2,001 sentences / 25,148 tokens.
- POS tags used for both training and parsing are gold UPOS tags, per the assignment spec (POS tagging itself is out of scope for this part).

3.4 Results

Metric	Score
Training-set transition accuracy	80.6%
Dev-set LAS	**56.84%**
Dev-set UAS	66.75%
Parsing speed	∼3,400 tokens/sec

Table 6: Dependency parser results.

3.5 Discussion

A LAS in the mid-50s is expected given the deliberately minimal feature set — only 4 POS tags, with no lexical/word-form features, distance features, or features of already-built arcs. Classic arc-standard parsers with richer feature templates (word forms, lemmas, children of the stack top, stack-buffer distance) typically reach 85–90+ LAS on this treebank. The gap here is

attributable almost entirely to feature poverty rather than a flawed transition system or oracle: the oracle reconstructs 100% of projective gold trees exactly (verified by asserting the oracle's own arc set matches the gold tree for every training sentence), so all evaluation error traces to the classifier's limited view of each configuration.

The parser is noticeably stronger on UAS than LAS (a ∼10-point gap), meaning it more often gets attachment approximately right but the label wrong — unsurprising since POS tags alone weakly signal finer-grained relations (e.g. `obj` vs. `obl`, `nsubj` vs. `csubj`). It also struggles most with long-distance attachments (e.g. PP-attachment ambiguity) and coordination, both classic hard cases for greedy, feature-light, beam-free parsers.

Straightforward extensions that would meaningfully raise LAS (not implemented, to stay within the assignment's specified feature set): word-form features for `s0`/`b0`; features of the already-built leftmost/rightmost children of `s0`/`s1`; and switching from greedy 1-best decoding to beam search.

3.6 Reproducing

```
git clone --depth 1 https://github.com/UniversalDependencies/UD_English-EWT.git
pip install scikit-learn numpy
python3 main.py        # trains, evaluates, and demos the 3 example sentences
```

Individual stages: `python3 train.py`, `python3 evaluate.py`, `python3 parser.py`.

4 Question 3 — Efficient Spelling Corrector

4.1 Overview

An end-to-end spelling correction system for both **non-word** errors (`sentnce` → `sentence`) and **real-word** errors (`I ate an apply` → `I ate an apple`) within edit distance 1, built on the NLTK Brown corpus:

File	Responsibility
`build_model.py`	One-shot script that builds and caches the model (`model.pkl`)
`src/model_builder.py`	Builds vocabulary, unigram counts, bigram counts, SymSpell-style deletes dictionary
`src/candidate_generation.py`	Brute-force edit-distance-1 generation and the symmetric-delete method
`src/corrector.py`	Non-word and real-word correction using n-gram context
`src/evaluate.py`	Benchmark test-set generation, accuracy, and speed comparisons
`src/cli.py`	Interactive terminal spelling-correction app

Table 7: Module responsibilities for the spelling corrector.

4.2 Design Choices

Corpus and language model. Built from the Brown corpus: a unigram frequency distribution, a bigram LM with add-one (Laplace) smoothing, and a deletion dictionary for fast candidate generation.

Candidate generation — two strategies for edit-distance-1 words:

1. **Method A (brute force):** generates all deletions, transpositions, substitutions, and insertions over the alphabet, then filters against the vocabulary.
2. **Method B (symmetric delete / SymSpell):** precomputes one-character deletions of every vocabulary word once, then at query time only computes the deletions of the query

word and looks them up in a hash map — moving the expensive alphabet-driven generation step out of the per-query hot path.

Correction logic. Non-word errors: rank all edit-distance-1 candidates by unigram frequency, pick the highest. Real-word errors: compare the bigram-context probability of the original phrase against candidate phrases, accepting a correction only when the candidate exceeds the original by a log-probability margin (`log_margin = 1.5` nats) — this avoids over-correcting in noisy or ambiguous contexts.

Evaluation strategy. Test cases are auto-generated by sampling 10% of Brown corpus sentences and injecting one single-edit typo per sentence, producing separate non-word and real-word benchmark sets.

Method B correctness. Raw symmetric-delete lookups can, in rare cases, match pairs that are genuinely edit-distance 2 apart (e.g. `at` and `to` both reduce to `t`); a final verification pass (`is_edit_distance_1`) ensures Method B's output always matches Method A's edit-distance-1 guarantee exactly.

4.3 Data

- Corpus: NLTK Brown corpus, auto-downloaded on first use and cached as `model.pkl`.
- Non-word test cases: 5,582. Real-word test cases: 3,713 (10% of Brown sentences, one typo per sentence).

4.4 Results

Metric	Score
Non-word correction accuracy	**81.03%**
Real-word correction accuracy	**68.87%**
Speed benchmark batch size	1,000 words
Method A runtime	0.0915 s
Method B runtime	0.0091 s
Speedup factor	**∼10.1x**

Table 8: Spelling corrector accuracy and speed benchmark.

4.5 Discussion

Method B is substantially faster because Method A must, for every word of length L, generate and hash roughly $54L + 25$ candidate strings (deletions, transpositions, 26 substitutions and 26 insertions per position) and test each for vocabulary membership — an alphabet-driven cost paid fresh on every query. Method B computes only the L one-character deletions of the query word and performs L lookups against a hash map that was already built once, up front, during model preparation. Because the expensive alphabet loop is moved entirely out of the per-query path, Method B trades a one-time $O(V \cdot \text{avg\_word\_len})$ preprocessing cost for an $O(L)$ per-query lookup cost, versus Method A's $O(26L)$ per-query generation cost — the empirically observed ∼10.1x speedup.

Real-word accuracy (68.87%) trails non-word accuracy (81.03%) because real-word correction is a harder, context-dependent disambiguation problem: the system must decide, from bigram evidence alone, whether an already-valid word is actually a mistake for a different valid word, and the required probability margin deliberately trades some recall for avoiding over-correction on noisy single-corpus bigram estimates.

Sample interactive session:

Input / Output	Latency
`> I hav a good feeling about this.` I **had** a good feeling about this.	1.44 ms
`> This is a test sentnce.` This is a test **sentence**.	1.83 ms
`> I would like to sea the world.` I would like to **see** the world.	1.71 ms

Table 9: Sample interactive spelling-correction session.

4.6 Conclusion

The corrector combines efficient candidate enumeration (SymSpell), corpus-driven frequency ranking, and contextual real-word correction via a bigram model, evaluated on automatically generated noisy samples. It balances accuracy and runtime: the model is precomputed once, and per-query latency is low enough for live, interactive use (see Question 4).

4.7 Reproducing

```
cd spelling_corrector
python -m venv .venv
.venv/Scripts/Activate
pip install -r requirements.txt
python build_model.py       # or let it build automatically on first run
python -m src.evaluate      # accuracy + Speed Demon benchmark
python -m src.cli           # live interactive CLI
```

5 Question 4 — Integrated Background Editor

5.1 Overview

A live, background-running editor that unifies Question 1's segmenter/POS tagger, Question 3's spelling corrector, and a newly implemented **PCFG constituency parser** (trained on a Penn Treebank sample) into a single real-time text-stream pipeline, exposed via a Streamlit web app and a CLI benchmark runner.

Newly implemented (Q4) components:

- `pcfg_parser.py` — PCFG induction in Chomsky Normal Form with a Viterbi/CKY chart parser and graceful failure handling, plus `BROWN_TO_PTB` tagset reconciliation.
- `language_model.py` — shared add-k smoothed bigram/trigram LM on the Brown corpus for sliding-window perplexity and sentence log-probability.
- `editor_engine.py` — live stream editor integrating Q1 and Q3.
- `app.py` — interactive Streamlit application (live typing, simulated typing stream, Speed Demon benchmark).
- `evaluator.py` / `run_benchmark.py` — latency benchmarking and sample-passage runs.

5.2 Pipeline Architecture

```
Incoming Stream (User Typing or Simulated Stream)
```

```
            |
            v
Fast-Typing Merges (spacebar drop probability p = 0.08)
            |
            v
Per-Token Live Pipeline
  1. Vocabulary check
  2. [SEGMENT-ALERT]  -- Q1 beam-search segmenter
  3. [SPELL-ALERT]    -- Q3 SymSpell / edit-distance-1
            |
            v
Periodic Trigger Pipeline (every N = 5 words)
  4. Sliding-window perplexity check
  5. Real-word error bigram context -> [GRAMMAR-ALERT]
            |
            v
End-of-Passage Analysis
  6. Sentence splitting
  7. PCFG CKY parse scoring
  8. Bigram / trigram sentence perplexity
  9. Multi-tier decision rule
```

5.3 Key Design Parameters & Justifications

- **Spacebar drop probability ($p = 0.08$):** natural touch-typing studies put spacebar omissions at roughly 5–10% of keyboarding slips; $p = 0.08$ produces about one merged token per 12–13 words — enough to stress-test the segmenter without producing gibberish.
- **Grammar trigger interval ($N = 5$ words):** too small ($N = 1$–2) causes excessive latency and false alarms from incomplete phrases lacking syntactic context; too large ($N \geq 10$) delays feedback. $N = 5$ roughly spans a full noun/verb phrase, balancing context sufficiency against imperceptible overhead.
- **Add-k smoothing ($k = 0.05$):** with Brown vocabulary $\approx$ 50,000 types, Laplace smoothing ($k = 1.0$) over-flattens the distribution; $k = 0.05$ (Lidstone) keeps genuinely ungrammatical sequences scoring high perplexity (> 650) while preserving discrimination for rare-but-valid collocations.
- **Tagset reconciliation:** Q1 emits Brown-style tags while the PCFG grammar (induced from Penn Treebank) requires PTB nonterminals. A deterministic `BROWN_TO_PTB` mapping strips morphological suffixes, maps functional equivalents (`AT`→`DT`, `NP`→`NNP`, `CS`→`IN`), and falls back to universal-POS mappings, preserving syntactic categories with $< 0.5\%$ ambiguity loss.
- **Method-selection decision rule:** a completed sentence is judged (1) **Grammatical (PCFG)** if the CKY parser finds a complete parse with normalized log-likelihood ≥ -8.5 bits/word; else (2) **Grammatical/Questionable (Trigram)** based on trigram perplexity thresholds ($\leq$ 400 / 400–800); else (3) **Acceptable/Ungrammatical (Bigram fallback)** at a 450 perplexity threshold.

5.4 Results

Real-time vs. end-of-passage agreement: across 20 multi-sentence test passages, real-time trigger alerts agreed with the final end-of-passage verdict on **86.4%** of sentences. Disagreements arise from two directions: a 5-word local window can miss long-distance subject-verb disagree-

ment (false negative), or an unusual opening phrase can spike local perplexity before the full sentence resolves grammatically (false positive).

PCFG vs. n-gram error detection: PCFG judgments excel at long-range structural violations (missing main verb, unclosed PPs, invalid coordination); n-gram judgments excel at local collocational abnormalities and semantic/selectional real-word errors that are syntactically well-formed (e.g. *"meat me at the station"* parses fine but is flagged by bigram transition probabilities).

Cascading error correction across sub-systems: merging `"themarket"` initially breaks PCFG parsing (`"unparseable"`); once Q1's segmenter splits it into *the* (DT) + *market* (NN), the `PP -> IN NP` rule resolves and a full parse succeeds. Similarly, the non-word `"sentnce"` scores near $-\infty$ under the trigram LM before correction; after Q3's SymSpell corrects it to `"sentence"`, trigram perplexity drops from 3,800 to 185, flipping the sentence's verdict from Ungrammatical to Grammatical (Trigram).

Speed Demon benchmark (1,000 corrupted tokens: 40% typos, 40% merges, 20% clean):

Metric	Full Live-Check (Seg + Spell)	Grammar-Only Check
Total latency (1,000 words)	**342.6 ms**	**48.1 ms**
Average latency per word	**0.343 ms**	**0.048 ms**
Latency added by Seg+Spell	+0.295 ms	—
Latency ratio	7.12x	1.0x

Table 10: Speed Demon benchmark results.

5.5 Discussion

Although the combined segmentation-plus-spelling layer is ∼7.1x slower than a pure n-gram lookup, 0.343 ms/word is orders of magnitude faster than human typing speed (∼150–250 ms per keystroke). Consequently the segmentation/spelling layer is cheap enough to run live on every token, while the grammar check remains throttled to $N = 5$ not for computational reasons but because syntactic and semantic plausibility genuinely requires a multi-word window to evaluate meaningfully.

Sample passage runs confirm the pipeline in practice: a six-sentence Gutenberg-style excerpt with 7 spacebar-drop merges (e.g. `'Thequick'`, `'overthe'`, `'oldman'`) is fully repaired and every sentence resolves to a PCFG-verified "Grammatical" verdict; a second passage containing a real-word error (`'meat'` → `'meet'`) and a non-word error (`'hav'` → `'have'`) is corrected token-by-token and likewise resolves to grammatical PCFG parses across all five sentences.

5.6 Summary of Key Achievements

- **Seamless integration:** 100% reuse of Question 1's segmenter/POS tagger and Question 3's candidate generation/corrector, with no duplicate model training.
- **Robustness:** gracefully handles unknown words, missing spacebars, typos, and homophone/real-word substitution errors.
- **Performance:** $<$ 0.4 ms average per-word processing time — well within the live-typing budget.
- **User experience:** a full Streamlit dashboard with real-time alert badges, interactive typing, simulated streaming, and structural analysis.

5.7 Reproducing

```
pip install -r background-editor/requirements.txt
streamlit run background-editor/app.py      # live web app
python background-editor/run_benchmark.py   # Speed Demon benchmark + sample runs
```

6 Cross-Cutting Discussion

Several themes recur across all four sub-projects:

1. **Segmentation quality dominates downstream accuracy.** Q1's error-source breakdown shows 61–71% of proposed-pipeline errors originate in segmentation rather than tagging, and Q4's cascading-error case studies show the same dynamic in a live setting (`"themarket"` breaking and then repairing PCFG parsing once correctly segmented). Investment in high-quality upstream segmentation pays compounding downstream dividends.
2. **Dynamic programming and global search beat greedy local decisions.** Q1's Viterbi segmenter avoids the greedy baseline's "trapping" failure mode; Q2's parser, by contrast, is explicitly greedy (1-best, no beam) and this is identified as its main avenue for improvement — reinforcing the same lesson from the opposite direction.
3. **Feature/parameter poverty is a more common failure mode than architectural flaws.** Q2's LAS gap versus richer parsers, and Q1's morphology-aware tagset underperforming due to sparsity rather than a wrong objective, both illustrate that additional expressiveness (features or fine-grained tags) is only beneficial when there is enough data to estimate it reliably.
4. **Efficiency engineering matters for real-time use.** Q3's SymSpell approach and Q4's tiered live/periodic/end-of-passage architecture both trade a one-time precomputation cost for cheap per-query/per-token work, which is what makes Q4's live editor practically usable at typing speed.
5. **Modular reuse compounds value.** Q4 does not retrain any Q1 or Q3 model; it reuses both directly and adds only the new PCFG and shared LM components, demonstrating that the earlier questions' interfaces were designed cleanly enough to integrate without modification.

7 Overall Conclusion

Across word segmentation, POS tagging, dependency parsing, spelling correction, and live-editor integration, statistically grounded models with proper smoothing and global (dynamic-programming or classifier-guided) search consistently and substantially outperform simple deterministic or frequency-based baselines — often by 2–7x on error-rate measures. The assignment's most consistent empirical finding is that **early-stage errors (segmentation, in particular) cascade disproportionately through downstream NLP stages**, which is borne out quantitatively in Question 1's error-source analysis and qualitatively in Question 4's live cascading-correction case studies. Where richer modeling was attempted (morphology-aware tagging, symmetric-delete candidate generation, PCFG-based grammar checking), the benefits were real but bounded by data sparsity and by the amount of context available at decision time — motivating the specific design choices (smoothing parameters, trigger intervals, decision thresholds) documented in each sub-report.

8 Reproducing All Systems

Question	Setup
Q1	`pip install -r requirements.txt` then `python main.py -eval-limit 400` (or `-lang english`/`-lang spanish`, or `-interactive`)
Q2	`git clone -depth 1 https://github.com/UniversalDependencies/UD_English-EWT.git && pip install scikit-learn numpy && python3 main.py`
Q3	`cd spelling_corrector && python -m venv .venv && pip install -r requirements.txt && python build_model.py && python -m src.evaluate` / `python -m src.cli`
Q4	`pip install -r background-editor/requirements.txt && streamlit run background-editor/app.py` (or `python background-editor/run_benchmark.py` for CLI benchmarks)

Table 11: Reproduction commands for all four systems.

Individual repositories retain their own `README.md` (usage/quickstart) and `REPORT.md` (full analysis) for reference; this document consolidates all four into a single deliverable.